



Module 05: Flask

Topics to be covered:

Introduction, Flask Environment Setup, App Routing, URL Building, Flask HTTP Methods, Flask Request Object, Flask cookies, File Uploading in Flask.

What is Web Framework?

Web Application Framework or simply Web Framework represents a collection of libraries and modules that enables a web application developer to write applications without having to bother about low-level details such as protocols, thread management etc.

WSGI

Web Server Gateway Interface (WSGI) has been adopted as a standard for Python web application development. WSGI is a specification for a universal interface between the web server and the web applications.

Werkzeug

It is a WSGI toolkit, which implements requests, response objects, and other utility functions. This enables building a web framework on top of it. The Flask framework uses Werkzeug as one of its bases.

Jinja2

Jinja2 is a popular templating engine for Python. A web templating system combines a template with a certain data source to render dynamic web pages.

Flask is often referred to as a micro framework. It aims to keep the core of an application simple yet extensible. Flask does not have built-in abstraction layer for database handling, nor does it have form validation support. Instead, Flask supports the extensions to add such functionality to the application. Some of the popular

In order to test **Flask** installation, type the following code in the editor as **Hello.py**



```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

Importing flask module in the project is mandatory. An object of Flask class is our **WSGI** application.

Flask constructor takes the name of **current module** (`__name__`) as argument.

The **route()** function of the Flask class is a decorator, which tells the application which URL should call the associated function.

```
app.route(rule, options)
```

- The **rule** parameter represents URL binding with the function.
- The **options** is a list of parameters to be forwarded to the underlying Rule object.

In the above example, '/' URL is bound with **hello_world()** function. Hence, when the home page of web server is opened in browser, the output of this function will be rendered.

Finally the **run()** method of Flask class runs the application on the local development server.

```
app.run(host, port, debug, options)
```

All parameters are optional



Sr.No.	Parameters & Description
1	host Hostname to listen on. Defaults to 127.0.0.1 (localhost). Set to '0.0.0.0' to have server available externally
2	port Defaults to 5000
3	debug Defaults to false. If set to true, provides a debug information
4	options To be forwarded to underlying Werkzeug server.

The above given **Python** script is executed from Python shell.

Python Hello.py

A message in Python shell informs you that

Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)

Open the above URL (**localhost:5000**) in the browser. '**Hello World**' message will be displayed on it.

Debug mode

A **Flask** application is started by calling the **run()** method. However, while the application is under development, it should be restarted manually for each change in the code. To avoid this inconvenience, enable **debug support**. The server will then reload itself if the code changes. It will also provide a useful debugger to track the errors if any, in the application.

The **Debug** mode is enabled by setting the **debug** property of the **application** object to **True** before running or passing the debug parameter to the **run()** method.

```
app.debug = True  
app.run()  
app.run(debug = True)
```



App Routing

Modern web frameworks use the routing technique to help a user remember application URLs. It is useful to access the desired page directly without having to navigate from the home page.

The **route()** decorator in Flask is used to bind URL to a function. For example –

```
@app.route('/hello')
def hello_world():
    return 'hello world'
```

Here, URL **'/hello'** rule is bound to the **hello_world()** function. As a result, if a user visits **http://localhost:5000/hello** URL, the output of the **hello_world()** function will be rendered in the browser.

The **add_url_rule()** function of an application object is also available to bind a URL with a function as in the above example, **route()** is used.

A decorator's purpose is also served by the following representation –

```
def hello_world():
    return 'hello world'
app.add_url_rule('/', 'hello', hello_world)
```

It is possible to build a URL dynamically, by adding variable parts to the rule parameter. This variable part is marked as **<variable-name>**. It is passed as a keyword argument to the function with which the rule is associated.

In the following example, the rule parameter of **route()** decorator contains **<name>** variable part attached to URL **'/hello'**. Hence, if the **http://localhost:5000/hello/Yaminee** is entered as a URL in the browser, **'TutorialPoint'** will be supplied to **hello()** function as argument.



```
from flask import Flask
app = Flask(__name__)

@app.route('/hello/<name>')
def hello_name(name):
    return 'Hello %s!' % name

if __name__ == '__main__':
    app.run(debug = True)
```

Save the above script as **hello.py** and run it from Python shell. Next, open the browser and enter URL **http://localhost:5000/hello/JOHN**.

The following output will be displayed in the browser.

Hello JOHN!

In addition to the default string variable part, rules can be constructed using the following converters –

Sr.No.	Converters & Description
1	int accepts integer
2	float For floating point value
3	path accepts slashes used as directory separator character



In the following code, all these constructors are used.

```
from flask import Flask
app = Flask(__name__)

@app.route('/blog/<int:postID>')
def show_blog(postID):
    return 'Blog Number %d' % postID

@app.route('/rev/<float:revNo>')
def revision(revNo):
    return 'Revision Number %f' % revNo

if __name__ == '__main__':
    app.run()
```

Run the above code from Python Shell. Visit the URL **<http://localhost:5000/blog/11>** in the browser.

The given number is used as argument to the **show_blog()** function. The browser displays the following output –

Blog Number 11

Enter this URL in the browser – **<http://localhost:5000/rev/1.1>**

The **revision()** function takes up the floating point number as argument. The following result appears in the browser window –

Revision Number 1.100000

The URL rules of Flask are based on **Werkzeug's** routing module. This ensures that the URLs formed are unique and based on precedents laid down by Apache.



Consider the rules defined in the following script –

```
from flask import Flask
app = Flask(__name__)

@app.route('/flask')
def hello_flask():
    return 'Hello Flask'

@app.route('/python/')
def hello_python():
    return 'Hello Python'

if __name__ == '__main__':
    app.run()
```

Both the rules appear similar but in the second rule, trailing slash (/) is used. As a result, it becomes a canonical URL. Hence, using /python or /python/ returns the same output. However, in case of the first rule, /flask/ URL results in **404 Not Found** page.

❖ Flask – Templates

It is possible to return the output of a function bound to a certain URL in the form of HTML. For instance, in the following script, **hello()** function will render **'Hello World'** with **<h1>** tag attached to it.

```
from flask import Flask
app = Flask(__name__)
```



```
@app.route('/')
def index():
    return '<html><body><h1>Hello World</h1></body></html>'

if __name__ == '__main__':
    app.run(debug = True)
```

However, generating HTML content from Python code is cumbersome, especially when variable data and Python language elements like conditionals or loops need to be put. This would require frequent escaping from HTML.

This is where one can take advantage of **Jinja2** template engine, on which Flask is based. Instead of returning hardcoded HTML from the function, a **HTML file can be rendered by the `render_template()` function.**

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def index():
    return render_template('hello.html')

if __name__ == '__main__':
    app.run(debug = True)
```

Flask will try to find the HTML file in the templates folder, in the same folder in which this script is present.

- Application folder
 - Hello.py
 - templates
 - hello.html

The term '**web templating system**' refers to designing an HTML script in which the variable data can be inserted dynamically. A web template system comprises of a template engine, some kind of data source and a template processor.

Flask uses **jinja2** template engine. A web template contains HTML syntax interspersed placeholders for variables and expressions (in these case Python expressions) which are replaced values when the template is rendered.



The following code is saved as **hello.html** in the templates folder.

```
<!doctype html>
<html>
  <body>

    <h1>Hello {{ name }}!</h1>

  </body>
</html>
```

Next, run the following script from Python shell.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<user>')
def hello_name(user):
    return render_template('hello.html', name = user)

if __name__ == '__main__':
    app.run(debug = True)
```

As the development server starts running, open the browser and enter URL as
– **http://localhost:5000/hello/mvl**

The **variable** part of URL is inserted at **{{ name }}** place holder.



The **jinja2** template engine uses the following delimiters for escaping from HTML.

- {% ... %} for Statements
- {{ ... }} for Expressions to print to the template output



- {# ... #} for Comments not included in the template output
- # ... ## for Line Statements

In the following example, use of conditional statement in the template is demonstrated. The URL rule to the **hello()** function accepts the integer parameter. It is passed to the **hello.html** template. Inside it, the value of number received (marks) is compared (greater or less than 50) and accordingly HTML is conditionally rendered.

The Python Script is as follows –

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/hello/<int:score>')
def hello_name(score):
    return render_template('hello.html', marks = score)

if __name__ == '__main__':
    app.run(debug = True)
```

HTML template script of **hello.html** is as follows –

```
<!doctype html>
<html>
  <body>
    {% if marks > 50 %}
      <h1> Your result is pass! </h1>
    {% else %}
      <h1> Your result is fail </h1>
    {% endif %}
  </body>
</html>
```

Note that the conditional statements **if-else** and **endif** are enclosed in delimiter **{%..%}**.

Run the Python script and visit URL **http://localhost/hello/60** and then **http://localhost/hello/30** to see the output of HTML changing conditionally.

The Python loop constructs can also be employed inside the template. In the following script, the **result()** function sends a dictionary object to template **results.html** when URL **http://localhost:5000/result** is opened in the browser.



The Template part of **result.html** employs a **for loop** to render key and value pairs of dictionary object **result{}** as cells of an HTML table.

Run the following code from Python shell.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/result')
def result():
    dict = {'phy':50,'che':60,'maths':70}
    return render_template('result.html', result = dict)

if __name__ == '__main__':
    app.run(debug = True)
```

Save the following HTML script as **result.html** in the templates folder.

```
<!doctype html>
<html>
<body>
  <table border = 1>
    {% for key, value in result.items() %}
      <tr>
        <th> {{ key }} </th>
        <td> {{ value }} </td>
      </tr>
    {% endfor %}
  </table>
</body>
</html>
```

Here, again the Python statements corresponding to the **For** loop are enclosed in `{%..%}` whereas, the expressions **key** and **value** are put inside `{{ }}`.

After the development starts running, open **http://localhost:5000/result** in the browser to get the following output.



❖ Flask – Request Object

The data from a client's web page is sent to the server as a global request object. In order to process the request data, it should be imported from the Flask module.

Important attributes of request object are listed below –

- **Form** – It is a dictionary object containing key and value pairs of form parameters and their values.
- **args** – parsed contents of query string which is part of URL after question mark (?).
- **Cookies** – dictionary object holding Cookie names and values.
- **files** – data pertaining to uploaded file.
- **method** – current request method.

Flask – Sending Form Data to Template

We have already seen that the http method can be specified in URL rule. The **Form** data received by the triggered function can collect it in the form of a dictionary object and forward it to a template to render it on a corresponding web page.

In the following example, '/' URL renders a web page (student.html) which has a form. The data filled in it is posted to the '/result' URL which triggers the **result()** function.

The **results()** function collects form data present in **request.form** in a dictionary object and sends it for rendering to **result.html**.

The template dynamically renders an HTML table of **form** data.

Given below is the Python code of application –



PARSHVANATH CHARITABLE TRUST'S

A. P. SHAH INSTITUTE OF TECHNOLOGY

Department of Information Technology

(NBA Accredited)



```
from flask import Flask, render_template, request
app = Flask(__name_)

@app.route('/')
def student():
    return render_template('student.html')

@app.route('/result', methods = ['POST', 'GET'])
def result():
    if request.method == 'POST':
        result = request.form
        return render_template("result.html", result = result)

if __name__ == '__main__':
    app.run(debug = True)
```

Given below is the HTML script of **student.html**.

```
<html>
<body>
  <form action = "http://localhost:5000/result" method = "POST">
    <p>Name <input type = "text" name = "Name" /></p>
    <p>Physics <input type = "text" name = "Physics" /></p>
    <p>Chemistry <input type = "text" name = "chemistry" /></p>
    <p>Maths <input type = "text" name = "Mathematics" /></p>
    <p><input type = "submit" value = "submit" /></p>
  </form>
</body>
</html>
```

Code of template (**result.html**) is given below –

```
<!doctype html>
<html>
<body>
  <table border = 1>
    {% for key, value in result.items() %}
      <tr>
        <th> {{ key }} </th>
        <td> {{ value }} </td>
      </tr>
```



```
{% endfor %}  
</table>  
</body>  
</html>
```

Run the Python script and enter the URL **http://localhost:5000/** in the browser.

localhost:5000

← → ↻ localhost:5000

Apps android java language live tv

Name

Physics

Chemistry

Maths

When the **Submit** button is clicked, form data is rendered on **result.html** in the form of HTML table.

localhost:5000/result

← → ↻ localhost:5000/result

Apps android java language live tv

Mathematics	70
chemistry	60
Physics	50
Name	MVL

❖ Flask – Cookies

A cookie is stored on a client's computer in the form of a text file. Its purpose is to remember and track data pertaining to a client's usage for better visitor experience and site statistics.

A **Request object** contains a cookie's attribute. It is a dictionary object of all the cookie variables and their corresponding values, a client has transmitted. In addition to it, a cookie also stores its expiry time, path and domain name of the site.

In Flask, cookies are set on response object. Use **make_response()** function to get response object from return value of a view function. After that, use the **set_cookie()** function of response object to store a cookie.



Reading back a cookie is easy. The **get()** method of **request.cookies** attribute is used to read a cookie.

In the following Flask application, a simple form opens up as you visit '/' URL.

```
@app.route('/')  
def index():  
    return render_template('index.html')
```

This HTML page contains one text input.

```
<html>  
  <body>  
    <form action = "/setcookie" method = "POST">  
      <p><h3>Enter userID</h3></p>  
      <p><input type = 'text' name = 'nm' /></p>  
      <p><input type = 'submit' value = 'Login' /></p>  
    </form>  
  </body>  
</html>
```

The Form is posted to '/setcookie' URL. The associated view function sets a Cookie name **userID** and renders another page.

```
@app.route('/setcookie', methods = ['POST', 'GET'])  
def setcookie():  
    if request.method == 'POST':  
        user = request.form['nm']  
  
        resp = make_response(render_template('readcookie.html'))  
        resp.set_cookie('userID', user)  
  
    return resp
```

'readcookie.html' contains a hyperlink to another view function **getcookie()**, which reads back and displays the cookie value in browser.

```
@app.route('/getcookie')  
def getcookie():  
    name = request.cookies.get('userID')  
    return '<h1>welcome '+name+'</h1>'
```

Run the application and visit **http://localhost:5000/**



The result of setting a cookie is displayed like this –



The output of read back cookie is shown below.



❖ Flask – File Uploading

Handling file upload in Flask is very easy. It needs an HTML form with its enctype attribute set to 'multipart/form-data', posting the file to a URL. The URL handler fetches file from `request.files[]` object and saves it to the desired location.



Each uploaded file is first saved in a temporary location on the server, before it is actually saved to its ultimate location. Name of destination file can be hard-coded or can be obtained from filename property of **request.files[file]** object. However, it is recommended to obtain a secure version of it using the **secure_filename()** function.

It is possible to define the path of default upload folder and maximum size of uploaded file in configuration settings of Flask object.

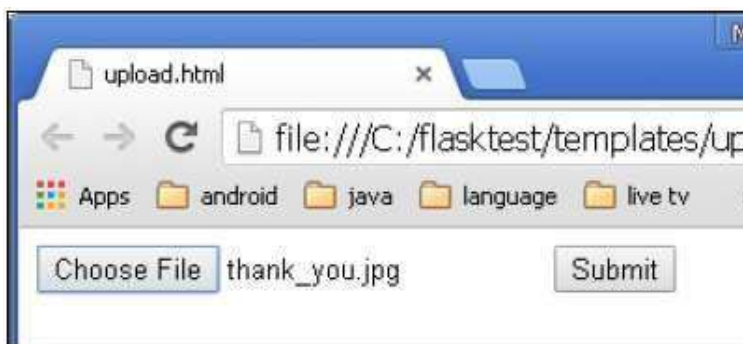
<code>app.config['UPLOAD_FOLDER']</code>	Defines path for upload folder
<code>app.config['MAX_CONTENT_PATH']</code>	Specifies maximum size of file to be uploaded – in bytes

The following code has **'upload'** URL rule that displays **'upload.html'** from the templates folder, and **'upload-file'** URL rule that calls **uploader()** function handling upload process.

'upload.html' has a file chooser button and a submit button.

```
<html>
<body>
  <form action = "http://localhost:5000/uploader" method = "POST"
    enctype = "multipart/form-data">
    <input type = "file" name = "file" />
    <input type = "submit"/>
  </form>
</body>
</html>
```

You will see the screen as shown below.





Click **Submit** after choosing file. Form's post method invokes '/upload_file' URL. The underlying function **uploader()** does the save operation.

Following is the Python code of Flask application.

```
from flask import Flask, render_template, request
from werkzeug import secure_filename
app = Flask(__name__)

@app.route('/upload')
def upload_file():
    return render_template('upload.html')

@app.route('/uploader', methods = ['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        f = request.files['file']
        f.save(secure_filename(f.filename))
        return 'file uploaded successfully'

if __name__ == '__main__':
    app.run(debug = True)
```